

Performance Analysis of a Thread Pool-Based Parallel Execution Model for Hybrid Post-Quantum TLS 1.3 Handshakes

Si-Woo Eum¹, Min-Ho Song¹, and Hwa-Jeong Seo¹

Hansung University, Seoul, South Korea
shuraatum@gmail.com, smino0906@gmail.com, hwajeong84@gmail.com

Abstract. The transition to post-quantum cryptography (PQC) significantly increases the computational cost of TLS 1.3 handshakes. In particular, hybrid handshakes incur even greater overhead, as they require performing both classical and PQC algorithms for key exchange and authentication. This paper systematically analyzes the performance of hybrid PQC TLS 1.3 handshakes using a POSIX thread pool-based parallel execution model. We evaluate a total of 135 combinations comprising 3 classical KEMs, 3 ML-KEM variants, 3 classical DSAs, and 5 PQC DSAs. Sequential execution times range from 429.2 to 1,907.0 μ s, while parallel execution times range from 356.7 to 1,380.8 μ s, achieving speedups of $1.08\times$ to $1.40\times$ across all combinations. The highest speedup is observed in P-384-based configurations, where the overlap between classical and PQC operations is most pronounced. Furthermore, we recommend both throughput-oriented combinations based on FN-DSA and currently standardized ML-DSA combinations for each NIST security level. These results provide practical design guidance for mitigating performance degradation in hybrid PQC TLS deployments.

Keywords: Post-Quantum Cryptography · TLS 1.3 · Hybrid Handshake · Thread-Level Parallelism · ML-KEM · ML-DSA

1 Introduction

Advances in quantum computing pose a fundamental threat to widely deployed public-key cryptographic systems. Shor’s algorithm [19] has shown that a sufficiently powerful quantum computer can break conventional algorithms such as RSA, ECDH, and ECDSA in polynomial time. To counter this threat, NIST has been driving the standardization of post-quantum cryptography (PQC), publishing ML-KEM [11], ML-DSA [10], and SLH-DSA [12] as final standards in 2024. NIST has also selected Falcon, the basis of FN-DSA, for ongoing standardization [13].

The TLS 1.3 protocol [17], a cornerstone of Internet communication security, is one of the most urgent areas requiring migration away from quantum-vulnerable algorithms. However, an immediate full replacement with post-quantum algorithms raises interoperability concerns and questions about the maturity

of new algorithms. Accordingly, current IETF Internet-Drafts specify a hybrid approach that simultaneously employs both classical and post-quantum algorithms [21].

A hybrid TLS handshake must execute pairs of classical and PQC algorithms for both key exchange (KEM) and digital signatures (DSA), substantially increasing computational overhead compared to a purely classical handshake. In particular, ML-DSA signing operations take hundreds of microseconds, and when combined with P-384-based ECDH/ECDSA, a single handshake can reach approximately 1.9 ms. This severely limits the number of TLS connections a server can process per second in high-scale environments.

This paper proposes a POSIX thread pool-based parallel execution model to mitigate this performance degradation and conducts a comprehensive performance benchmark across diverse hybrid algorithm combinations. The main contributions of this work are as follows:

- Design and implementation of a four-stage hybrid TLS 1.3 handshake model, where the independent classical and PQC operations within each stage are parallelized at the thread level.
- A systematic benchmark and analysis of 135 total combinations: 3 classical KEMs $\times$ 3 PQC KEMs $\times$ 3 classical DSAs $\times$ 5 PQC DSAs (including 3 ML-DSA and 2 FN-DSA variants).
- Factor analysis of parallelization speedups ranging from $1.08\times$ to $1.40\times$, along with optimal combination recommendations for each NIST security level.

The remainder of this paper is organized as follows. Section 2 describes the background on PQC standards, TLS 1.3 handshakes, and hybrid composition. Section 3 presents the design of the four-stage parallel hybrid handshake model. Section 4 describes the experimental setup and methodology. Section 5 presents the performance analysis results for all 135 combinations. Section 6 concludes the paper and discusses future work.

2 Background and Related Work

2.1 Post-Quantum Cryptography Standards

NIST has been conducting a PQC standardization process since 2016 [1]. It published ML-KEM [11] as the KEM standard and ML-DSA [10] and SLH-DSA [12] as signature standards in 2024. Falcon, the basis of FN-DSA, has been selected for ongoing standardization [13].

ML-KEM is based on CRYSTALS-Kyber [3] and provides three parameter sets: ML-KEM-512, ML-KEM-768, and ML-KEM-1024, corresponding to NIST security Levels 1, 3, and 5, respectively. Level 1 targets security equivalent to AES-128, while Level 5 targets security equivalent to AES-256.

For signature algorithms, ML-DSA and FN-DSA are the benchmarking targets in this study. ML-DSA is based on CRYSTALS-Dilithium [5] and provides

ML-DSA-44, ML-DSA-65, and ML-DSA-87, corresponding to Levels 2, 3, and 5, respectively. FN-DSA is based on Falcon [6] and provides FN-DSA-512 at Level 1 and FN-DSA-1024 at Level 5. The two algorithms exhibit distinct trade-offs. FN-DSA produces significantly smaller signatures—approximately 666 B for FN-DSA-512, which is about 3.6 times smaller than ML-DSA-44’s approximately 2,420 B—making it advantageous in bandwidth-sensitive environments. FN-DSA-512 also exhibits lower signing times compared to ML-DSA-44. How these characteristics affect the parallelization benefits in hybrid TLS handshakes is analyzed in Section 5.

These post-quantum algorithms functionally replace their classical counterparts. For key exchange, ML-KEM replaces the role of ECDH, while for digital signatures, ML-DSA and FN-DSA replace the roles of RSA and ECDSA. However, ECDH is a key agreement primitive where both parties exchange ephemeral public keys and independently compute a shared secret, whereas ML-KEM operates as a KEM primitive where one party encapsulates and the other decapsulates. Therefore, integrating these two primitives in a hybrid key exchange requires interface-level abstraction [18].

To address this, our study applies the standard technique of converting Diffie–Hellman key exchange into a KEM [2]. During encapsulation, an ephemeral key pair is generated, a Diffie–Hellman operation is performed with the counterpart’s public key to derive a shared secret, and the ephemeral public key is transmitted as ciphertext. During decapsulation, the received ciphertext—i.e., the counterpart’s ephemeral public key—is used with one’s own secret key to perform the same Diffie–Hellman operation and recover the shared secret. This conversion is mathematically equivalent to the original ECDH key exchange, and the same approach is adopted in the X25519MLKEM768 hybrid key exchange currently being standardized by the IETF [21]. As a result, both classical and post-quantum algorithms are abstracted into the unified KEM interface (KEYGEN, ENCAPS, DECAPS), simplifying hybrid composition and parallelization modeling.

2.2 TLS 1.3 Handshake Protocol

TLS 1.3 [17] is a protocol that reduces handshake round trips and strengthens security compared to previous versions. During the 1-RTT handshake, the client and server establish a shared secret through key exchange and verify the server’s identity through the digital signature in the server certificate. The computational cost of the handshake is concentrated in key exchange and signature verification, and this cost directly depends on the cryptographic algorithms used.

Figure 1 illustrates the points at which cryptographic operations are performed during a TLS 1.3 1-RTT handshake. The client sends a **ClientHello** message containing a public key for key exchange, and the server uses it in **ServerHello** to establish a shared secret. The server then transmits a certificate and **CertificateVerify** message over the encrypted channel, generating a digital signature over the handshake transcript. The client verifies this signature using the public key from the received certificate to confirm the server’s identity.

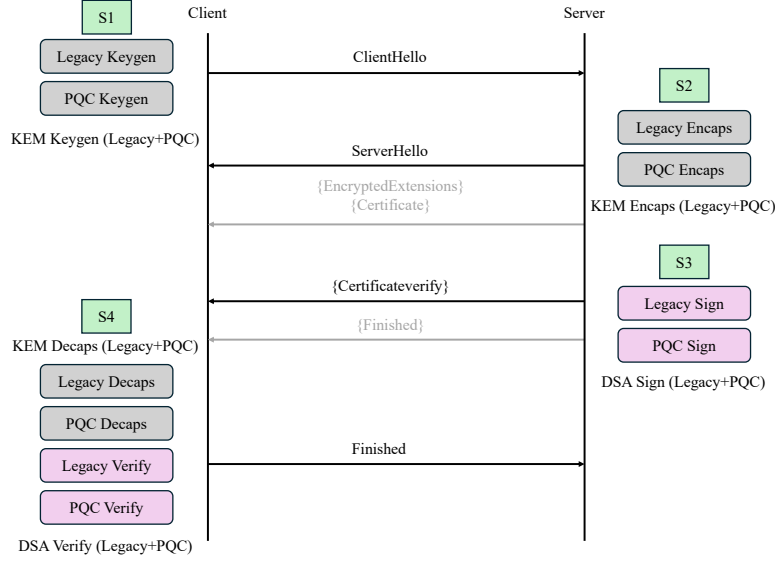


Fig. 1: Cryptographic operation structure of a hybrid TLS 1.3 1-RTT handshake (S1–S4).

In summary, the computationally intensive stages of the handshake are divided into two parts—key exchange (KEM/ECDH) and digital signatures (DSA)—and this study analyzes the performance impact of configuring both operations in a hybrid manner.

2.3 Hybrid Composition

A hybrid TLS handshake leverages both classical and post-quantum algorithms for key exchange and authentication, ensuring that security is maintained even if one of the two is compromised [21]. This approach has the advantage of defending against “harvest-now, decrypt-later” attacks during the transition period before the quantum threat materializes, while retaining the proven security of classical algorithms [9].

Hybrid composition can be broadly classified into two types depending on how the two algorithms are combined. For key exchange, **concatenation-based combination** independently executes two KEMs and concatenates their respective shared secrets to derive the final key as $ss = \text{KDF}(ss_{\text{legacy}} || ss_{\text{pqc}})$; this is the approach described in the current IETF Internet-Draft [21]. In contrast, **nesting-based combination** uses the output of one algorithm as the input to the other, creating a sequential dependency [7]. For authentication, similarly, **composite signatures** independently generate two signatures over the same message and transmit them as a concatenated pair [15], while **nested signatures** embed one signature within the input of the other.

In prior work, Paquin et al. [16] conducted PQC benchmarks in TLS but focused primarily on hybrid KEMs without addressing the parallelization of hybrid authentication. Sikeridis et al. [20] analyzed PQC authentication performance in TLS 1.3 but only considered sequential execution. Crockett et al. [4] implemented a prototype of hybrid key exchange; however, comprehensive combination analysis with both KEM and DSA under hybrid composition and parallelization effects remained insufficiently studied.

2.4 Multi-Core Parallelism and Thread Pools

Modern CPUs are equipped with multiple physical cores, enabling improved throughput by executing independent operations simultaneously across cores. A prominent mechanism for leveraging such multi-core parallelism is POSIX threads (pthread), a multithreading API defined by IEEE Std 1003.1 [8] and the de facto standard on UNIX-like operating systems. Each pthread is created as a kernel thread and assigned to an individual core by the operating system scheduler, enabling true parallel execution.

However, creating and destroying threads on a per-request basis incurs repeated costs for context allocation and stack memory provisioning. For tasks with execution times on the order of hundreds of microseconds—such as cryptographic operations—this overhead becomes relatively significant. A thread pool pre-creates and maintains a fixed number of worker threads, distributing tasks through a work queue. This amortizes the creation and destruction costs to a one-time initialization expense, minimizing the delay from task submission to execution start. In this study, we adopt this thread pool pattern as the parallelization foundation for the hybrid handshake; the specific application is described in Section 3.2.

3 Parallel Hybrid Handshake Model

3.1 Four-Stage Handshake Structure

The 1-RTT handshake of TLS 1.3 [17] consists of three message flights from ClientHello and ServerHello through Finished. In this study, we decompose this message flow into four stages based on **cryptographic operation units** rather than protocol layers. Each stage corresponds to a point in the handshake where cryptographic operations are concentrated, and the key premise for parallelization is that the classical and PQC algorithm operations within each stage have no data dependencies. Furthermore, by applying the DH-to-KEM conversion described in Section 2.3, the classical key exchange ECDH and the post-quantum key exchange ML-KEM are abstracted into the same KEM interface (KEYGEN, ENCAPS, DECAPS).

Stage 1: ClientHello (Key Generation). The client generates ephemeral key pairs to prepare the `key_share` extension for the ClientHello message. Both the classical KEM key generation and PQC KEM key generation are performed in this stage, and the two operations have no mutual dependencies.

Stage 2: ServerHello-KEM (Key Encapsulation). During the generation of the `key_share` response in `ServerHello`, the server performs key encapsulation against the client’s public key. The classical KEM encapsulation and PQC KEM encapsulation can be executed independently.

Stage 3: ServerHello-Sign (Signature Generation). The server generates a digital signature over the handshake transcript for the `CertificateVerify` message. The classical DSA signature and PQC DSA signature are independently performed over the same transcript. In a real TLS server, the signing key associated with the certificate is pre-existing; accordingly, our model pre-generates the server signing key to reflect a realistic scenario.

Stage 4: ClientFinish (Verification and Completion). After receiving the server’s response, the client recovers the shared secret and verifies the signature. Specifically, four operations are performed: classical KEM decapsulation, PQC KEM decapsulation, classical DSA signature verification, and PQC DSA signature verification. All four operations are mutually independent.

3.2 Thread Pool-Based Parallelization Strategy

To execute the independent operations within each handshake stage in parallel, we employ a pthread-based thread pool. The thread pool pre-creates and maintains a fixed number of worker threads (4 by default), managing tasks through a circular queue synchronized via mutexes and condition variables.

The parallel execution model operates as follows:

1. At the start of each stage, the independent operations for that stage are submitted to the work queue via `thread_pool_submit()`.
2. Worker threads retrieve tasks from the queue and execute them in parallel.
3. `thread_pool_wait()` is called to wait until all tasks for that stage are completed (barrier synchronization).
4. Execution proceeds to the next stage.

The number of parallelizable tasks varies by stage: Stages 1 and 2 each have 2 tasks, Stage 3 has 2 tasks, and Stage 4 has up to 4 independent tasks that can be executed in parallel. Consequently, the greatest speedup is expected in Stage 4.

3.3 Algorithm Combinations

The combination space of algorithms evaluated in this study is summarized in Table 1. We exhaustively evaluate a total of 135 combinations: 3 classical KEMs $\times$ 3 PQC KEMs $\times$ 3 classical DSAs $\times$ 5 PQC DSAs. The security level of each algorithm is categorized along classical and quantum dimensions, and the effective security level of a hybrid composition is determined by the minimum of the two dimensions.

Table 1: Hybrid algorithm combinations and NIST security levels.

Type	Algorithm	Classical Security	PQC Level
KEM (Classical)	X25519	~128-bit	–
KEM (Classical)	ECDH P-256	~128-bit	–
KEM (Classical)	ECDH P-384	~192-bit	–
KEM (PQC)	ML-KEM-512	–	Level 1
KEM (PQC)	ML-KEM-768	–	Level 3
KEM (PQC)	ML-KEM-1024	–	Level 5
DSA (Classical)	Ed25519	~128-bit	–
DSA (Classical)	ECDSA P-256	~128-bit	–
DSA (Classical)	ECDSA P-384	~192-bit	–
DSA (PQC)	ML-DSA-44	–	Level 2
DSA (PQC)	ML-DSA-65	–	Level 3
DSA (PQC)	ML-DSA-87	–	Level 5
DSA (PQC)	FN-DSA-512	–	Level 1
DSA (PQC)	FN-DSA-1024	–	Level 5

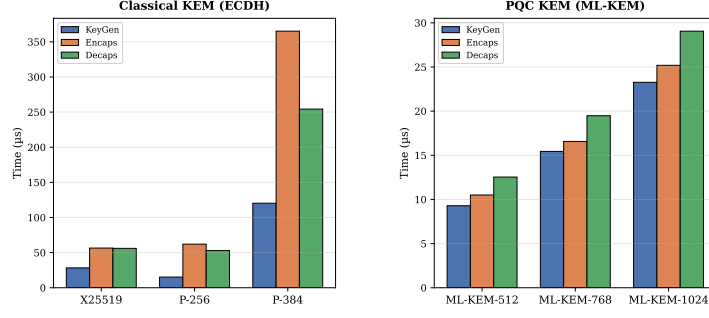
Table 2: Experimental environment configuration.

Component	Specification
CPU	Apple M2 (8 cores: 4P + 4E)
OS	macOS (Darwin 25.2.0)
Compiler	Apple Clang 17.0.0
Optimization	-O2
OpenSSL	3.6.0
liboqs	0.15.0
Thread pool	4 worker threads

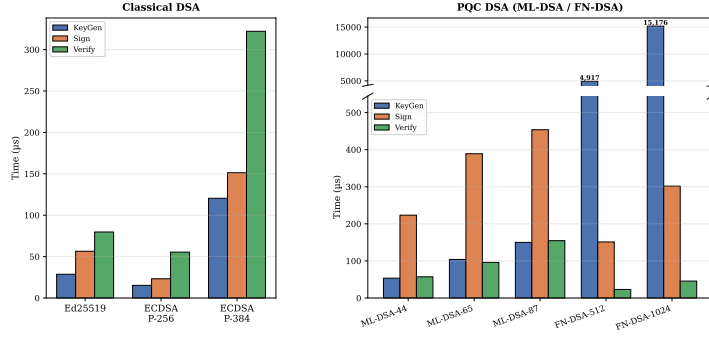
4 Experimental Setup and Methodology

Table 2 summarizes the experimental environment. All experiments were conducted on identical hardware and software configurations. Classical cryptographic algorithms including X25519, ECDH, ECDSA, and Ed25519 were implemented through the EVP interface of OpenSSL 3.6.0 [23], while post-quantum algorithms including ML-KEM, ML-DSA, and FN-DSA were implemented using liboqs 0.15.0 [14]. Background on the OQS project is provided in [22].

To ensure benchmark accuracy and reproducibility, the following methodology was applied. For each algorithm combination, 100 warm-up iterations were performed followed by 1,000 measured iterations, with execution times recorded in microseconds (μs) and averaged.



(a) KEM operation (KeyGen, Encaps, Decaps) execution times.



(b) DSA operation (KeyGen, Sign, Verify) execution times.

Fig. 2: Individual operation execution times (μs) for each hybrid KEM and DSA combination.

The following additional techniques were applied to improve measurement reliability:

- The measurement order of sequential and parallel handshakes was alternated between odd and even iterations to minimize bias from cache state.
- In individual operation benchmarks, the return values of cryptographic operations were checked at every iteration, and failed samples were excluded from the statistics. In handshake benchmarks, a failed operation immediately aborted that iteration, as a handshake failure indicates a fundamental correctness issue.
- Signing keys were pre-generated, consistent with the practice in real TLS servers.

5 Results and Analysis

5.1 Individual Algorithm Performance

Figure 2 shows the execution times of individual operations for each hybrid KEM and DSA combination.

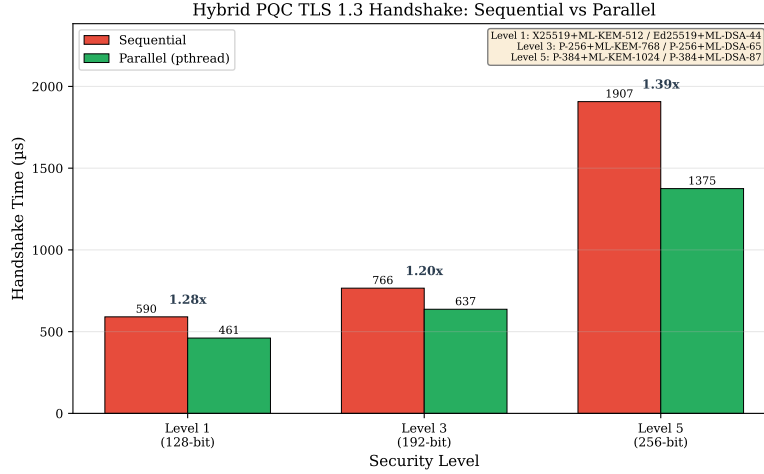


Fig. 3: Comparison of sequential and parallel handshake times (μs) for representative combinations at each NIST security level.

For KEM operations, P-384-based combinations exhibit the slowest performance due to the high computational cost of ECDH. The encapsulation time for P-384 is 365.0–365.3 μs , approximately 6 times slower than P-256’s 62.0–62.1 μs and X25519’s 56.3–56.7 μs . In contrast, the PQC KEM ML-KEM is relatively fast across all variants, with even ML-KEM-1024 decapsulation taking only 29.1 μs .

For DSA operations, PQC signatures are the dominant cost component. ML-DSA-87’s signing time is 450.2–455.6 μs , which is 8 to 20 times that of Ed25519’s 56.5 μs or ECDSA P-256’s 23.2 μs . The classical DSA with P-384 also incurs substantial costs at 151.3 μs for signing and 322.1 μs for verification, maximizing the hybrid overhead in P-384 + ML-DSA-87 combinations.

FN-DSA-based combinations exhibit a clear signing performance advantage over ML-DSA. FN-DSA-512’s signing time is approximately 151 μs , about 32% faster than ML-DSA-44’s approximately 223 μs , and its verification time is approximately 23 μs , less than half of ML-DSA-44’s approximately 57 μs . FN-DSA-1024 achieves signing in approximately 302 μs and verification in approximately 46 μs , showing a substantial advantage over ML-DSA-87’s signing time of approximately 454 μs . However, FN-DSA key generation is very slow—approximately 4,917 μs for FN-DSA-512 and approximately 15,176 μs for FN-DSA-1024—but since real TLS servers pre-generate signing keys, this does not affect handshake throughput.

5.2 Handshake Performance Comparison

Figure 3 compares the sequential and parallel handshake times for representative combinations at each NIST security level. The sequential execution time across

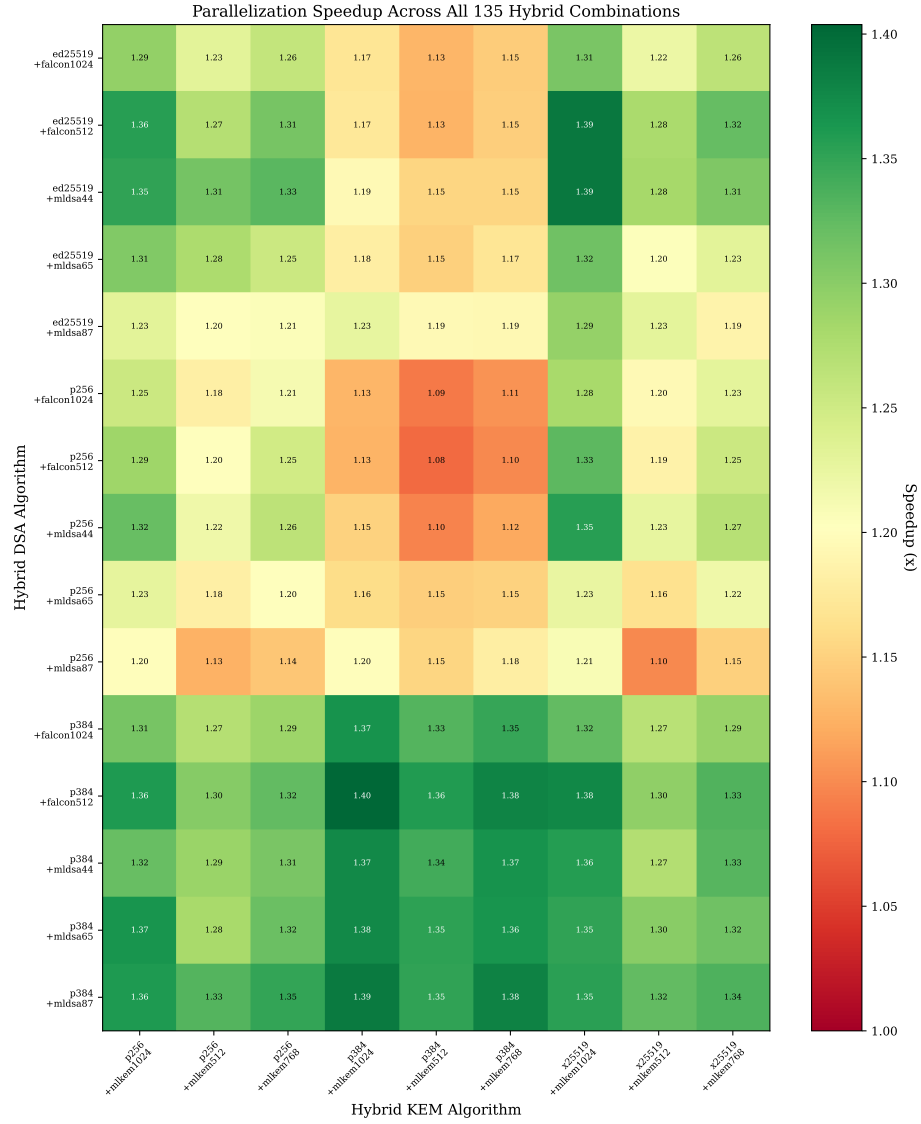


Fig. 4: Parallelization speedup heatmap for all 135 hybrid combinations (rows: hybrid DSA, columns: hybrid KEM).

all 135 combinations ranges from 429.2 to 1,907.0 μs , and the parallel execution time ranges from 356.7 to 1,380.8 μs .

As the security level increases, handshake times grow; however, the absolute time savings from parallelization also increase. The Level 1 representative combination X25519 + ML-KEM-512 + Ed25519 + ML-DSA-44 is reduced from 590.5 μs sequentially to 460.8 μs in parallel, saving 129.7 μs and achieving a $1.28\times$ speedup. The Level 5 representative combination P-384 + ML-KEM-1024 + P-384 + ML-DSA-87 is reduced from 1,907.0 μs to 1,375.0 μs , saving 532.0 μs and achieving a $1.39\times$ speedup, confirming that parallelization benefits are more pronounced at higher security levels.

5.3 Parallelization Speedup Analysis

Figure 4 visualizes the speedup across all 135 combinations as a heatmap. The overall speedup ranges from a minimum of $1.08\times$ to a maximum of $1.40\times$.

The most prominent pattern in the heatmap is the high speedup of P-384-based combinations. Combinations using P-384 for both KEM and DSA achieve speedups of $1.33\times$ – $1.40\times$, because the high computational cost of classical P-384 operations overlaps effectively with the PQC operations. In particular, FN-DSA-based combinations exhibit faster signing operations compared to ML-DSA, resulting in a higher ratio of parallel overlap with classical operations; the P-384 + ML-KEM-1024 + P-384 + FN-DSA-512 combination achieves the maximum speedup of $1.40\times$. However, the effective PQC security level of this combination is limited to Level 1 by FN-DSA-512. Conversely, when using P-256 or X25519 with P-256 as the classical DSA, the classical operations are significantly faster than the PQC operations, limiting the benefits of parallelization.

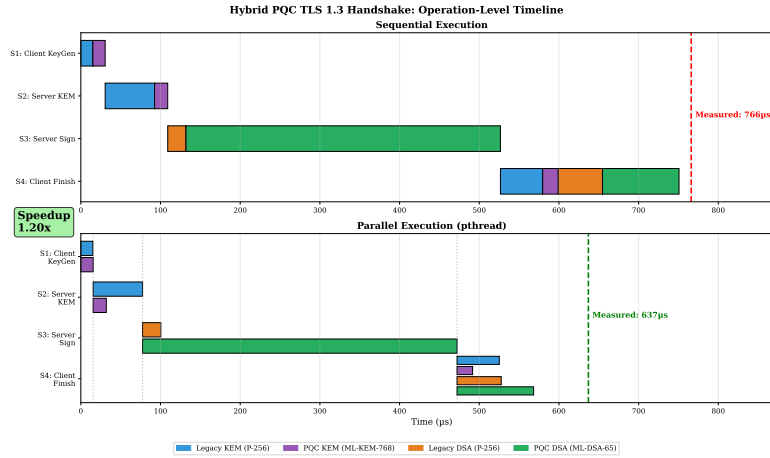


Fig. 5: Per-operation timeline for the Level 3 representative combination (P-256+ML-KEM-768 / P-256+ML-DSA-65). Top: sequential; Bottom: parallel.

Figure 5 shows the per-operation timeline for the Level 3 representative combination P-256 + ML-KEM-768 / P-256 + ML-DSA-65. In sequential execution, the four stages run sequentially for a total of 766 μ s, while in parallel execution, classical and PQC operations within each stage proceed concurrently, reducing the total to 637 μ s. The parallelization effect is most pronounced in Stage 4 (ClientFinish), where four independent operations are parallelized, reducing the sequential 228 μ s to 129 μ s in parallel and achieving a per-stage speedup of 1.76 $\times$.

Table 3: Recommended hybrid combinations by NIST security level.

Category	Level	Recommended Combination (KEM / DSA)	Seq. (μ s)	Par. (μ s)	Speedup
FN-DSA	Level 1	P-256 + ML-KEM-512 / P-256 + FN-DSA-512	429.2	356.7	1.20 $\times$
FN-DSA	Level 5	P-384 + ML-KEM-1024 / P-384 + FN-DSA-1024	1,658.6	1,213.0	1.37 $\times$
ML-DSA	Level 1	P-256 + ML-KEM-512 / P-256 + ML-DSA-44	532.7	436.9	1.22 $\times$
ML-DSA	Level 3	P-256 + ML-KEM-768 / P-256 + ML-DSA-65	766.0	637.0	1.20 $\times$
ML-DSA	Level 5	P-384 + ML-KEM-1024 / P-384 + ML-DSA-87	1,907.0	1,375.0	1.39 $\times$

5.4 Recommended Combinations by Security Level

Table 3 presents the recommended combinations for each NIST security level. The recommendation criteria are based on the combination that achieves the fastest handshake time under parallel execution within each security level.

FN-DSA-based combinations achieve faster handshake times than ML-DSA-based combinations at the same security level. At Level 1, the P-256 + ML-KEM-512 + P-256 + FN-DSA-512 combination delivers the fastest parallel handshake at 356.7 μ s, corresponding to approximately 2,803 handshakes per second. The ML-DSA-based Level 1 recommended combination achieves 436.9 μ s, processing approximately 2,289 handshakes per second—a throughput difference of approximately 22% compared to the FN-DSA-based combination. At Level 5, P-384-based combinations are essential. The FN-DSA-1024-based combination can operate at 1,213.0 μ s with parallelization, corresponding to approximately 824 handshakes per second. The ML-DSA-87-based combination can also process approximately 727 handshakes per second at 1,375.0 μ s, completing within the millisecond range and demonstrating practical deployability.

6 Conclusion

This paper proposes a POSIX thread pool-based parallelization model for hybrid post-quantum TLS 1.3 handshakes and conducts a comprehensive performance analysis across 135 algorithm combinations, including 3 ML-DSA and 2 FN-DSA variants.

The key findings are summarized as follows. First, thread-level parallelization achieves speedups of $1.08\times$ to $1.40\times$ over sequential execution across all 135 combinations. Second, the highest speedups of $1.33\times$ to $1.40\times$ are observed in high-security configurations using P-384 for both KEM and DSA, as the high computational cost of classical algorithms overlaps effectively with PQC operations. Third, ranging from the fastest parallel handshake of $356.7\ \mu\text{s}$ with FN-DSA-512 to $1,375.0\ \mu\text{s}$ for the highest security level, all combinations complete within approximately 1.4 ms, confirming practical deployability.

These results demonstrate that thread-level parallelization is a practical strategy for effectively mitigating performance degradation during the hybrid PQC transition, without requiring additional hardware investment, within existing multi-core server environments. In particular, since the parallelization benefit is greatest at higher security levels, it serves as the most useful optimization measure precisely where the performance burden of the PQC transition is heaviest.

However, this study is limited to cryptographic operation benchmarks on a single platform, the Apple M2, and does not account for the network I/O, certificate parsing, and other overhead present in actual TLS handshakes. Furthermore, since an independent benchmark environment was used, performance may differ from that within actual TLS libraries.

Future work includes combining algorithm-level parallelization using SIMD instructions such as ARM NEON and x86 AVX2/AVX-512 with thread-level parallelization, cross-platform benchmarks on diverse server hardware, and integration into production TLS libraries such as OpenSSL and BoringSSL for end-to-end performance evaluation in network environments.

References

1. Alagic, G., Apon, D., Cooper, D., Dang, Q., Dang, T., Kelsey, J., Lichtinger, J., Liu, Y.K., Miller, C., Moody, D., Peralta, R., Perlner, R., Robinson, A., Smith-Tone, D.: Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process. Internal Report NISTIR 8413, NIST (2022). <https://doi.org/10.6028/NIST.IR.8413-upd1>
2. Barnes, R., Bhargavan, K., Lipp, B., Wood, C.A.: Hybrid Public Key Encryption. RFC 9180 (2022). <https://doi.org/10.17487/RFC9180>
3. Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS – Kyber: A CCA-Secure Module-Lattice-Based KEM. In: IEEE European Symposium on Security and Privacy (EuroS&P). pp. 353–367. IEEE (2018). <https://doi.org/10.1109/EuroSP.2018.00032>

4. Crockett, E., Paquin, C., Stebila, D.: Prototyping Post-Quantum and Hybrid Key Exchange and Authentication in TLS and SSH. *IACR Cryptology ePrint Archive* **2019**, 858 (2019)
5. Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS – Dilithium: A Lattice-Based Digital Signature Scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)* **2018**(1), 238–268 (2018). <https://doi.org/10.13154/tches.v2018.i1.238-268>
6. Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z.: Falcon: Fast-Fourier Lattice-Based Compact Signatures over NTRU. In: *NIST Post-Quantum Cryptography Standardization Round 3 Submissions* (2020)
7. Giacon, F., Heuer, F., Poettering, B.: KEM Combiners. In: *Public-Key Cryptography (PKC 2018)*. LNCS, vol. 10769, pp. 190–218. Springer (2018). https://doi.org/10.1007/978-3-319-76578-5_7
8. IEEE/The Open Group: IEEE Standard for Information Technology—Portable Operating System Interface (POSIX) Base Specifications, Issue 7. *IEEE Std 1003.1-2017*, IEEE (2018). <https://doi.org/10.1109/IEEESTD.2018.8277153>
9. Mosca, M.: Cybersecurity in an Era with Quantum Computers: Will We Be Ready? *IEEE Security & Privacy* **16**(5), 38–41 (2018). <https://doi.org/10.1109/MSP.2018.3761723>
10. National Institute of Standards and Technology: Module-Lattice-Based Digital Signature Standard (FIPS 204). *Federal Information Processing Standards Publication 204*, NIST (2024). <https://doi.org/10.6028/NIST.FIPS.204>
11. National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203). *Federal Information Processing Standards Publication 203*, NIST (2024). <https://doi.org/10.6028/NIST.FIPS.203>
12. National Institute of Standards and Technology: Stateless Hash-Based Digital Signature Standard (FIPS 205). *Federal Information Processing Standards Publication 205*, NIST (2024). <https://doi.org/10.6028/NIST.FIPS.205>
13. National Institute of Standards and Technology: NIST Post-Quantum Cryptography Project. <https://csrc.nist.gov/projects/post-quantum-cryptography> (2026), accessed: 2026-03-03
14. Open Quantum Safe: Release liboqs version 0.15.0. <https://github.com/open-quantum-safe/liboqs/releases/tag/0.15.0> (2025), accessed: 2026-03-03
15. Ounsworth, M., Gray, J., Pala, M., Klaussner, J.: Composite ML-DSA for use in X.509 Public Key Infrastructure. *Internet-Draft, draft-ietf-lamps-pq-composite-sigs-15* (work in progress) (2026)
16. Paquin, C., Stebila, D., Tamvada, G.: Benchmarking Post-Quantum Cryptography in TLS. In: *Post-Quantum Cryptography (PQCrypto 2020)*. LNCS, vol. 12100, pp. 72–91. Springer (2020). https://doi.org/10.1007/978-3-030-44223-1_5
17. Rescorla, E.: The Transport Layer Security (TLS) Protocol Version 1.3. *RFC 8446* (2018). <https://doi.org/10.17487/RFC8446>
18. Schwabe, P., Stebila, D., Wiggers, T.: ML-KEM Post-Quantum Key Agreement for TLS 1.3. *Internet-Draft, draft-ietf-tls-mlkem-07* (work in progress) (2026)
19. Shor, P.W.: Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In: *Proceedings 35th Annual Symposium on Foundations of Computer Science*. pp. 124–134. IEEE (1994). <https://doi.org/10.1109/SFCS.1994.365700>
20. Sikeridis, D., Kampanakis, P., Devetsikiotis, M.: Post-Quantum Authentication in TLS 1.3: A Performance Study. In: *Network and Distributed System Security Symposium (NDSS 2020)* (2020). <https://doi.org/10.14722/ndss.2020.24203>

21. Stebila, D., Fluhrer, S., Gueron, S.: Hybrid Key Exchange in TLS 1.3. Internet-Draft, draft-ietf-tls-hybrid-design-16 (work in progress) (2025)
22. Stebila, D., Mosca, M.: Post-Quantum Key Exchange for the Internet and the Open Quantum Safe Project. In: Selected Areas in Cryptography (SAC 2016). LNCS, vol. 10532, pp. 14–37. Springer (2017)
23. The OpenSSL Project Authors: OpenSSL 3.6 Series Release Notes. <https://www.openssl.org/news/openssl-3.6-notes/> (2025), accessed: 2026-03-03